SWAM: Scalable Event-Driven Incident Management System

Technical Report

Muhammad Sohail

SOFTWARE ARCHITECTURES AND METHODOLOGIES 2025-2026

https://github.com/msohailse/SWAM

Contents

1 Objective and Purpose of the Project Work

The proposed web application is an event-driven system designed to manage incident creation and automatically deduplicate redundant reports. Every incident is routed to a specific department, which holds exclusive responsibility for its resolution. Incidents are categorized by dynamic tags (e.g., fire or theft) that serve as lookup values. Although administrators typically manage the master list of tags, users can create new categories directly from the incident-creation view if a tag is missing. To facilitate clear communication, reporters and department staff can collaborate via a dedicated commenting system on each issue.
The system is structured around the following core components and architectural patterns:

- **Frontend:** Developed as an Angular Single Page Application.
- **Backend Architecture:**
  - **Hexagonal Architecture:** the core domain is isolated behind ports and adapters, exposing a REST API built with Jakarta EE, JAX RS, CDI, and JPA on Quarkus with PostgreSQL.
  - **Lightweight CQRS:** supports flexible read side queries without affecting the write path or requiring a separate projection store.
  - **Testing:** uses Testcontainers for integration tests (verifying department isolation) and unit tests for domain logic (e.g., email validation).
- **Event Driven Architecture:** the backend publishes incident creation events to a Kafka topic; an independent analyzer service consumes these events to deduplicate and reports back via a second topic, which the backend consumes to mark duplicates. This flow is verified by the integration test `DuplicateDetectorTest`.

2 Software Requirements Analysis

This section specifies the software requirements in accordance with ISO/IEC/IEEE 29148:2011. Modal verbs follow RFC 2119: **SHALL** denotes a mandatory requirement, **SHOULD** a recommended one, **MUST NOT** a prohibition, and **MAY** an optional feature. Each requirement is atomic (one verifiable capability) and grouped by responsible actor or subsystem.

2.1 Functional Requirements

2.1.1 Reporter Requirements

- **FR1: User Registration:** The system **SHALL** register a new user given a first name, last name, email, and password.
- **FR2: Default Role Assignment:** A newly registered user **SHALL** be assigned the default role `REPORTER`.
- **FR3: Unique Registration Email:** The system **MUST NOT** register two users with the same email address.
- **FR4: User Authentication:** The system **SHALL** authenticate a user by matching email and password.
- **FR5: Authentication Failure Handling:** The system **SHALL** reject a login attempt if the email does not exist or the password does not match.

- **FR6: Scope Enforcement:** The system **SHALL** limit the incident list to show only the incidents reported by the acting user if their role is `REPORTER`.
- **FR7: Incident Creation Input:** A user **SHALL** be able to create a new incident by providing a non-blank title, severity, and a tag name; the description is optional.
- **FR8: Unassigned Incident Department:** During incident creation, the system **SHALL** leave the assigned department empty by default.
- **FR9: Existing Tag Reuse:** During incident creation, if the specified tag already exists, the system **SHALL** reuse it.
- **FR40: Automatic Tag Creation:** During incident creation, if the specified tag does not already exist, the system **SHALL** create it automatically.
- **FR10: Incident Modification:** A `REPORTER` **SHALL** be able to update the title, description, and severity of their own incidents.
- **FR11: Reassignment Restriction:** The system **MUST NOT** allow a `REPORTER` to assign or change the department of any incident.
- **FR12: Thread Commenting:** A `REPORTER` **SHALL** be able to add comments to any incident they are permitted to view.
- **FR36: Incident Deletion:** A `REPORTER` **SHALL** be able to delete their own incidents.

2.1.2 Department Admin Requirements

- **FR13: Scope Enforcement:** The system **SHALL** filter the listed incidents to only display those assigned to the acting user's department if the acting user is an active `ADMIN`.
- **FR14: Close Boundary:** A department `ADMIN` **SHALL** only close incidents assigned to their own department.
- **FR15: Assigned Department Close Restriction:** The system **MUST NOT** allow closing an incident that has no assigned department.
- **FR16: Incident Modification:** A department `ADMIN` **SHALL** be able to update the title, description, and severity of any incident assigned to their department.
- **FR17: Reassignment:** A department `ADMIN` **SHALL** be able to assign or reassign the department of any incident assigned to their department.
- **FR18: Manage Tag:** Both `SUPER_ADMIN` and `ADMIN` **SHALL** be able to create, update, and delete tags globally; tags have no department scope.
- **FR19: Thread Commenting:** A department `ADMIN` **SHALL** be able to add comments to incidents assigned to their department.
- **FR37: Incident Deletion:** A department `ADMIN` **SHALL** be able to delete incidents assigned to their own department.

2.1.3 Super Admin Requirements

- **FR20: Create Admins:** A `SUPER_ADMIN` **SHALL** be permitted to create a user of any type, including `REPORTER`, `ADMIN`, or another `SUPER_ADMIN`.
- **FR21: Admin Department Mapping:** Both a department `ADMIN` and a `SUPER_ADMIN` **SHALL** be able to create a new `ADMIN` and assign it to exactly one Department at creation time; a department `ADMIN`'s assignment is forced to their own department, while a `SUPER_ADMIN` may choose any department.
- **FR22: Admin Expiry Configuration:** A `SUPER_ADMIN` **MAY** assign an expiration date (in days) to a newly created `ADMIN`'s role.
- **FR23: Expired Admin Privilege Revocation:** Once an `ADMIN`'s role has expired, the system **SHALL** revoke their ability to close incidents and to assign or reassign departments.
- **FR39: Expired Admin Scope Reversion:** Once an `ADMIN`'s role has expired, the system **SHALL** apply the same reporter-scoped incident filtering as FR6 to that user.
- **FR24: Unscoped Visibility:** The system **SHALL** display all incidents across all departments without restriction if the acting user is a `SUPER_ADMIN`.
- **FR25: Close Incident:** A `SUPER_ADMIN` **SHALL** be permitted to close any incident in the system.
- **FR26: Close Reassignment:** Any active `ADMIN` or `SUPER_ADMIN` **SHALL** be permitted to assign or reassign a department to an incident during the closure action, same as during a plain update.
- **FR27: Global Modification:** A `SUPER_ADMIN` **SHALL** be permitted to update the title, description, severity, and department of any incident in the system.
- **FR28: Manage Department:** A `SUPER_ADMIN` **SHALL** be able to create, update, and delete departments with unique names and descriptions.
- **FR29: Duplicate Department Restriction:** The system **MUST NOT** allow two departments with the same name.
- **FR30: Thread Commenting:** A `SUPER_ADMIN` **SHALL** be able to add comments to any incident in the system.
- **FR38: Incident Deletion:** A `SUPER_ADMIN` **SHALL** be able to delete any incident in the system.

2.1.4 System / Analyzer Integration Requirements

- **FR31: Async Creation Messaging:** Upon incident creation, the system **SHALL** publish an `incident-created` event to a Kafka topic.
- **FR32: Event Ingestion:** The Analyzer Service **SHALL** consume events from the `incident-created` Kafka topic.
- **FR33: Fuzzy Duplicate Detection:** The Analyzer **SHALL** detect potential duplicates among open incidents using fuzzy text similarity checking with a threshold > 0.4.

- **FR34: Event Production:** When a duplicate is detected, the Analyzer Service **SHALL** publish an `AnalysisCompletedEvent` containing the incident and duplicate IDs to a secondary Kafka topic.
- **FR35: Async Duplicate Resolve:** The API service **SHALL** consume the `AnalysisCompletedEvent` from Kafka and transactionally update the incident's duplicate flag and append a system-authored comment.

2.2 Non-Functional Requirements

2.2.1 Scalability and Elasticity

- **NFR1: Independent Scalability:** The `api-service` and `analyzer-service` **SHALL** be independently scalable via separate Kubernetes Deployments.
- **NFR2: API Autoscaling:** A Horizontal Pod Autoscaler **SHALL** scale the `api-service` between 1 and 5 replicas, targeting 50% average CPU utilization.
- **NFR3: API Pod Resources:** Each backend pod **SHALL** request 200m CPU / 384Mi memory with limits of 500m CPU / 512Mi memory.

2.2.2 Containerization and Deployment

- **NFR4: Local Containerization:** All services (frontend, backend, analyzer, PostgreSQL, Kafka) **SHALL** be containerized and orchestrated via Docker Compose for local development.
- **NFR5: K8s Declarative Manifests:** The same services **SHALL** be deployable to a Kubernetes cluster using declarative manifests in a dedicated `swam` namespace.
- **NFR6: Secure Credentials Injection:** Database credentials **SHALL** be injected via environment variables (Kubernetes Secrets in production); the system **MUST NOT** hardcode credentials in source code.
- **NFR7: Profile-Aware Configuration:** Application configuration **SHALL** support profile-aware overrides (`%dev`, `%test`, `%prod`) within a single configuration file.

2.2.3 Service Independence

- **NFR8: Service Decoupling:** The `api-service` and `analyzer-service` **SHALL** share no Java source code.
- **NFR9: Decoupled Communications:** Inter-service communication **SHALL** occur exclusively via Kafka topics (`incident-created`, `incident-analysis-completed`).
- **NFR10: Non-Blocking Analysis:** Duplicate detection **SHALL** happen asynchronously without blocking the incident creation write path.

2.2.4 Transactional Integrity and Error Handling

- **NFR11: Transactional Write Operations:** All write operations (creation, update, closure, commenting) **SHALL** execute within a single `@Transactional` database transaction.
- **NFR12: Global Exception Mapper:** Domain-level validation failures **SHALL** return HTTP `400 Bad Request` with a JSON error body via a global exception mapper.

- **NFR13: Secure Exception Handling:** The system **MUST NOT** expose HTTP `500 Internal Server Error` for business-rule violations.

2.2.5 Testability

- **NFR14: Domain Validation Coverage:** Unit tests **SHALL** cover domain validation logic (e.g., email format, password strength, empty fields).
- **NFR15: Dev Services Integration Tests:** Integration tests **SHALL** use Testcontainers (via Quarkus Dev Services) to verify persistence and REST endpoints against a real PostgreSQL instance.
- **NFR16: Security Policy Integration Tests:** Integration tests **SHALL** verify department-scoped access control (e.g., a department admin cannot close another department's incident).

2.2.6 Security

- **NFR17: Password Quality Validation:** The system **SHALL** enforce a minimum password length of 8 characters, including at least one digit and one uppercase letter.
- **NFR18: Email Format Validation:** The system **SHALL** validate that email addresses contain an `@` symbol and a dot before persisting.
- **NFR19: Username Validation:** The system **SHALL** validate that user names contain no whitespace characters.
- **NFR20: Mandatory Closing Comment:** The system **SHALL** validate that the closing comment text is non-empty and non-blank when an incident is closed.

2.3 Data Requirements

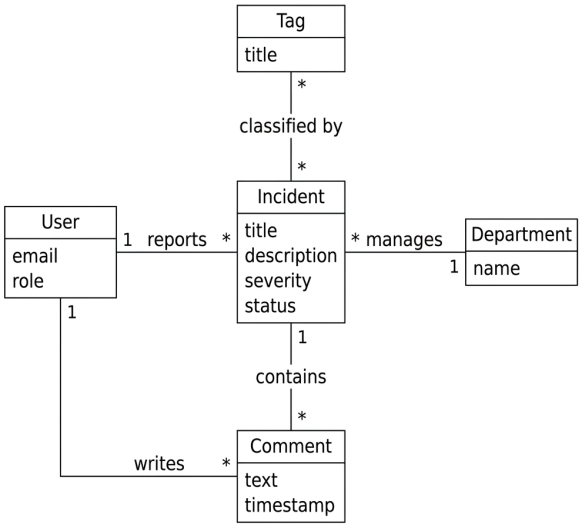
The logical data model centres on the following entities and their relationships:

- **User:** identified by a unique email; holds a role (`REPORTER`, `ADMIN`, `SUPER_ADMIN`), an optional department association, and an optional admin expiry timestamp.
- **Incident:** requires a non-blank title, severity, a reporter (User), and a tag; optionally assigned to a Department. Tracks closure status and duplicate flag.
- **Tag:** a category (e.g., "Fire", "Theft", "Network") used to classify incidents.
- **Department:** an organizational unit (e.g., "IT", "Maintenance") to which incidents can be assigned and that scopes department admins.
- **Comment:** belongs to exactly one Incident and one User (author); supports threaded conversation.

3 Conceptual Model

The conceptual model serves as a technology-independent dictionary that defines the core business concepts used throughout the use cases. The following dictionary defines the core business concepts used throughout the use cases.

User	Any human interacting with the system (Reporter, Department Admin, or Super Admin).
Incident	The central entity representing a reported problem that needs to be tracked and resolved.
Department	A logical organizational grouping responsible for managing specific types of incidents.
Tag	A flexible classification label attached to incidents.
Comment	A text entry made by a user to collaborate on an incident's resolution.



4 Use Case Analysis

Four actors interact with the system: the **Reporter**, the **Department Admin**, the **Super Admin**, and the **Analyzer** (an autonomous system actor).

- **Reporter:** Can register, login, report incidents, view & filter their own incidents, reply to comment threads, and update their own reported incidents.
- **Department Admin:** Inherits all Reporter capabilities, plus closing department incidents, updating department incidents, reassigning departments for department incidents, and managing tags.
- **Super Admin:** Inherits all Department Admin capabilities, plus viewing & filtering all incidents globally, closing any incident, reassigning department fields globally, managing departments, and creating administrative users.
- **Analyzer (System):** Autonomously detects duplicates based on similarity and publishes duplicate completed events.

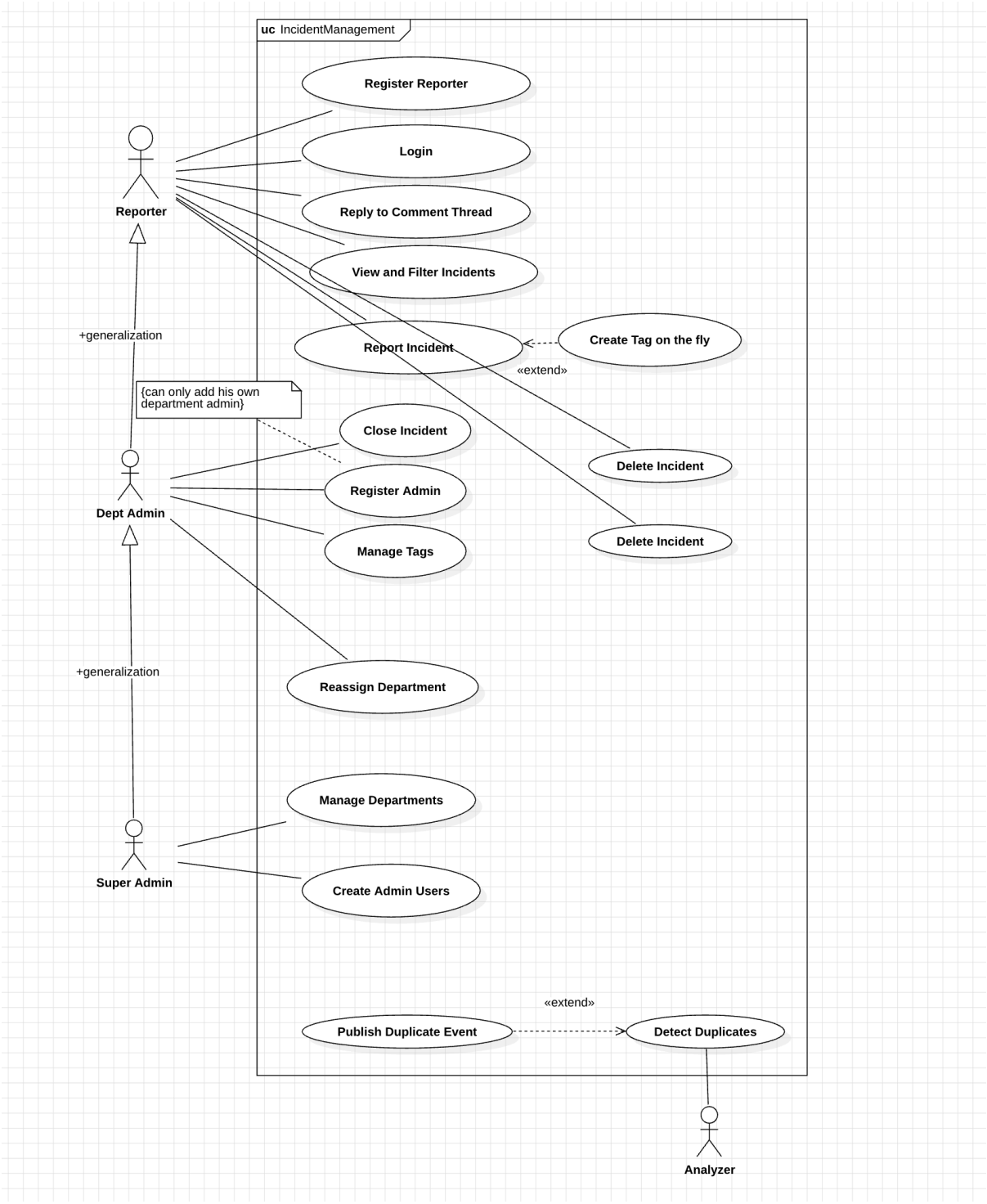


Figure 1. UML Use Case Diagram

4.1 Detailed Use Case Descriptions

Use Case	**Report Incident**
Actor	Reporter (inherited by Department Admin and Super Admin via generalization).
Level	User Goal.
Basic Course	1. The Actor selects the option to report a new incident. 2. The system presents the incident creation form. 3. The Actor fills in the title and severity (both mandatory), an optional description, and either picks an existing tag or types a new one. 4. The Actor submits the form. 5. The system validates the input (FR7), resolves the tag or creates it automatically if it doesn't exist yet (FR9, FR40), persists the incident with no department assigned (FR8), and returns the created incident to the Actor.
Postcondition	An `incident-created` event is published to Kafka, triggering the asynchronous duplicate-detection pipeline (§9.6) without blocking this use case's response.

Use Case	**Retrieve Incidents (View & Filter)**
Actor	Reporter, Department Admin, or Super Admin — one shared use case whose result set depends on the Actor's own role.
Level	User Goal.
Basic Course	1. The Actor opens the Incidents screen. 2. The system resolves the Actor's scope from their own `User` record, never from a client-supplied parameter: • **Reporter** → sees only incidents they personally reported (FR6). • **Expired Admin** (time-boxed grant lapsed) → reverts to reporter-scoped visibility, seeing only incidents they personally reported (FR39). • **Department Admin** (active) → sees only incidents assigned to their own department (FR13). • **Super Admin** → sees every incident in the system, unscoped (FR24). 3. The Actor may optionally narrow the already-scoped list further with tag, severity, and/or status filters. 4. The system returns the filtered, scope-restricted list.
Note	Because the scope always comes from the Actor's own record, a request can narrow what it sees but never widen it — this is the CQRS-Lite read path detailed in the Detailed Design section.

Use Case	**Reassign Department**
Actor	Department Admin or Super Admin.
Level	User Goal.
Precondition	The Actor is an active Admin; a Reporter can never perform this use case regardless of whose incident it is (FR11).
Basic Course	1. The Actor opens an incident they are permitted to update — their own department's incident, or (for a Super Admin) any incident. 2. The Actor selects a different Department from the dropdown, or clears it. 3. The Actor saves the change, either as a plain edit (FR17) or as part of closing the incident (FR26 — both entry points now enforce the same active-Admin rule). 4. The system verifies the Actor's admin status, applies the new department assignment, and persists it.
Postcondition	If a Department Admin reassigned the incident away from their own department, it no longer appears in that Actor's own Retrieve Incidents result set on the next query — the same scoping rule applies uniformly, with no special-case removal logic.

Use Case	**Create Department**
Actor	Super-Admin (only).
Level	User Goal.
Precondition	Acting user must be a Super-Admin (validated by 'User.isSuperAdmin()' in 'DepartmentService.create').
Basic Course	1. The Super-Admin selects "Create Department" in the admin UI (FR28). 2. The system presents a form requiring a unique department name and optional description (FR28, FR29). 3. The Super-Admin fills in the fields and submits the form. 4. The backend validates the name's uniqueness (FR29), checks the author's role (FR28), persists a new 'Department' entity, and returns it.
Postcondition	A new department appears in the department list and can be used for incident assignment (FR28).

Use Case	**Add Comment**
Actor	Reporter, Department Admin, Super Admin
Level	User Goal.

Basic Course	1. The Actor opens an incident they have permission to view. 2. The Actor enters a non-blank comment text and submits. 3. The system validates the comment is non-empty (FR12 for Reporter, FR19 for Admin, FR30 for Super Admin) and persists it linked to the incident.
Postcondition	The comment becomes part of the incident's thread and is visible to all actors with view permission.

Use Case	**Delete Incident**
Actor	Reporter, Department Admin, Super Admin
Level	User Goal.
Basic Course	1. The Actor selects an incident they are allowed to delete. 2. The system checks permission (FR36 for Reporter, FR37 for Admin, FR38 for Super Admin) and removes the incident and its comments.
Postcondition	The incident and all its associated data are no longer present in the system.

Use Case	**Register User**
Actor	Department Admin or Super Admin
Level	User Goal.
Basic Course	1. The Actor selects "Create User" in the admin UI. 2. The system presents a form for first name, last name, email, password, and role. 3. The Actor fills the form and submits — a Department Admin's role choice is limited to REPORTER/ADMIN (scoped to their own department for ADMIN), while a Super Admin may additionally create another SUPER_ADMIN (FR20/FR21). 4. The system validates uniqueness of email (FR3) and password rules (NFR17), creates the user with the selected role, and returns the new user record.
Postcondition	A new user account exists and can log in according to its role.

Use Case	**Login**
Actor	Reporter, Department Admin, Super Admin
Level	User Goal.
Basic Course	1. The Actor provides email and password. 2. The system verifies credentials (FR4) and, on success, issues a session token. 3. On failure, the system returns an authentication error (FR5).

Postcondition	The Actor obtains an authenticated session for subsequent actions.

Use Case	**Close Incident (with optional reassignment)**
Actor	Department Admin, Super Admin
Level	User Goal.
Basic Course	1. The Actor opens an incident they are permitted to close. 2. Optionally, the Actor selects a new Department (or clears) before closing. 3. The Actor provides a mandatory closing comment (NFR20). 4. The system validates admin status (FR13/FR24) and comment non-empty (NFR20), updates incident status to closed, records the comment, and persists any department change (FR26).
Postcondition	The incident is marked closed, the closing comment is stored, and if reassigned, the new department takes effect.

4.2 Temporal Constraints and Expiration Rules

The system enforces temporal rules on roles. The `Department Admin` actor is subjected to an optional expiration constraint:

- **adminExpiresAt constraint:** Administrative use cases (such as closing incidents, filtering by department, and managing tags) have a precondition that the current system time is less than the user's `adminExpiresAt` timestamp (or that the timestamp is null).
- **Graceful Degradation:** If the system clock exceeds the expiration timestamp, the user's state changes to expired. They can still perform base `Reporter` actions (viewing own reports, replying to comments) but any attempt to execute administrative use cases will be blocked by the system's authorization validation.

5 Domain Modeling

The domain layer implements the core business concepts introduced in the conceptual model: User, Incident, Department, Tag, and Comment. These entities are persisted as JPA entities with unidirectional @ManyToOne relationships, ensuring a simple and robust mapping to the relational database.

- **User**: Represents any human interacting with the system (Reporter, Department Admin, or Super Admin). Stores email, role, optional department, and optional admin expiration.
- **Incident**: The central entity for a reported problem, containing title, description, severity, status, and links to its reporter, assigned department, and associated tag.
- **Department**: Organizational grouping responsible for managing assigned incidents.
- **Tag**: Flexible classification label attached to incidents.
- **Comment**: Text entry made by a user to collaborate on an incident's resolution.

The domain design applies key course analysis idioms:

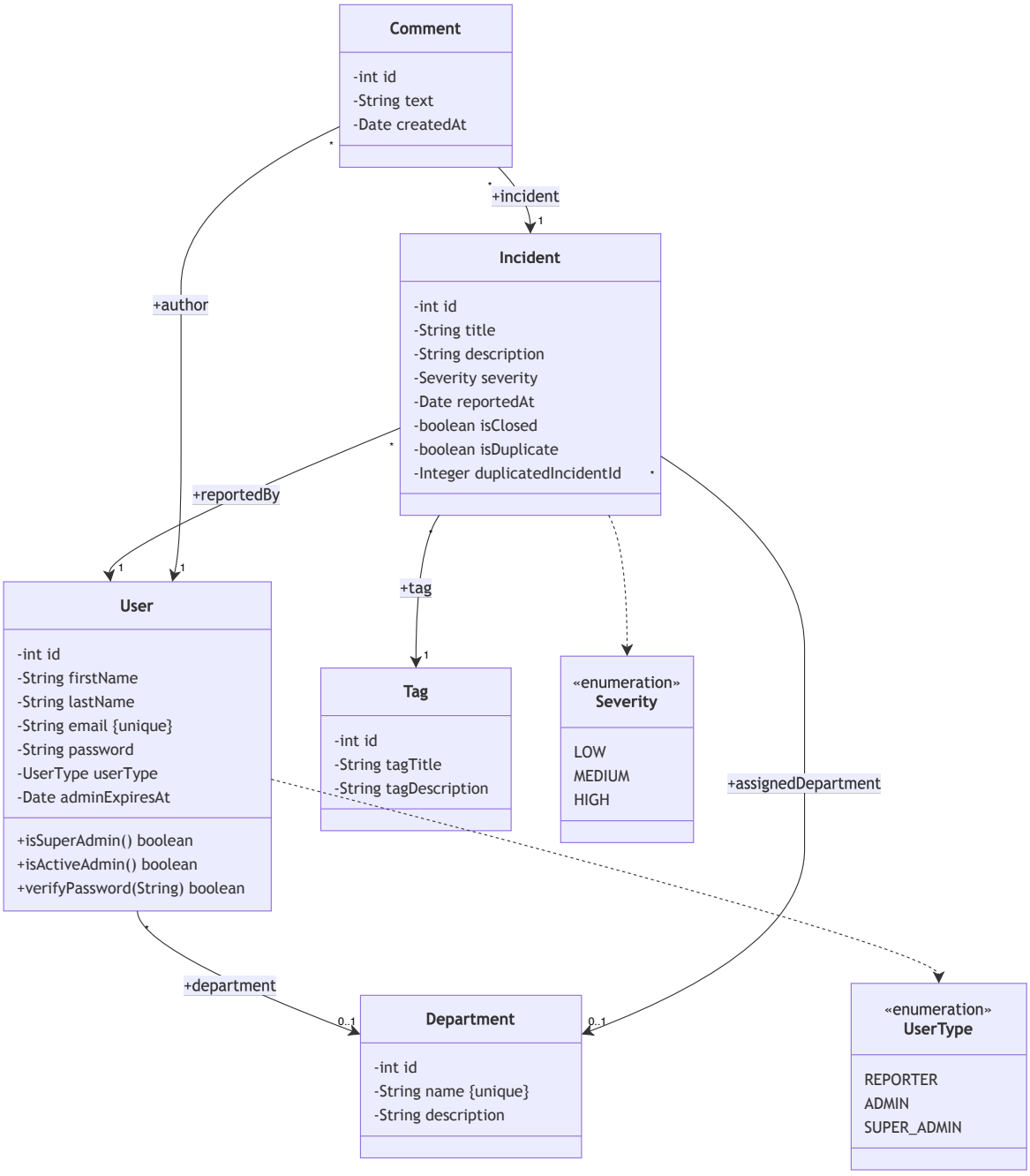


Figure 1: UML Class Diagram for the Domain Model

- **Roles are Lighter than Types:** Rather than subclassing `User` into `Reporter`, `DepartmentAdmin`, or `SuperAdmin`, role differentiation is represented by the `UserType` enumeration and the optional `adminExpiresAt` date field. This prevents inheritance hierarchies from becoming stiff and keeps the domain model highly flexible.
- **Avoiding Target/Agent Anti-Pattern:** Operations and setters are associated directly with the target objects whose state changes (such as name validation within `User` and duplicate-flag tracking within `Incident`) rather than the user acting as an agent. External orchestration (e.g., matching similarity or checking access boundaries) is deferred to the stateless service and adapter layers.
- **Clean Unidirectional Associations:** All database relations are mapped unidirectionally from child entities to parent/lookup entities (e.g., `Incident` referencing its reporter `User`, assigned `Department`, and `Tag`), mirroring the exact JPA '@ManyToOne' bindings in the codebase. This simplifies mapping and avoids runtime circular references.

6 Application Architecture

Each service follows a **Pragmatic Hexagonal** layering approach.

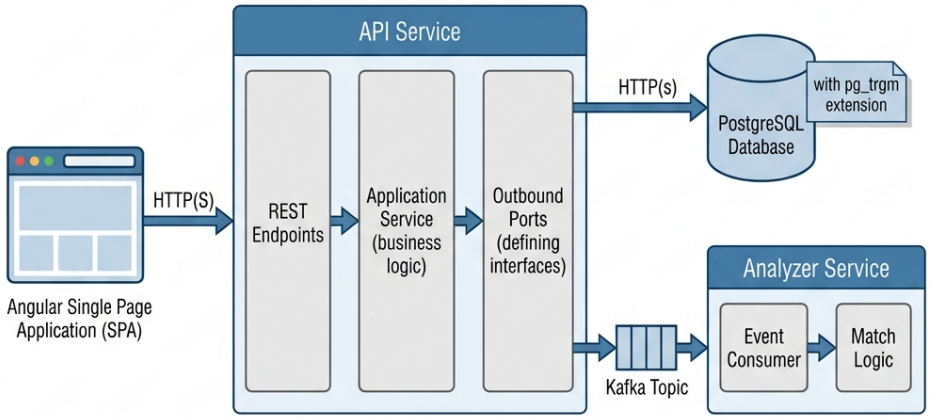
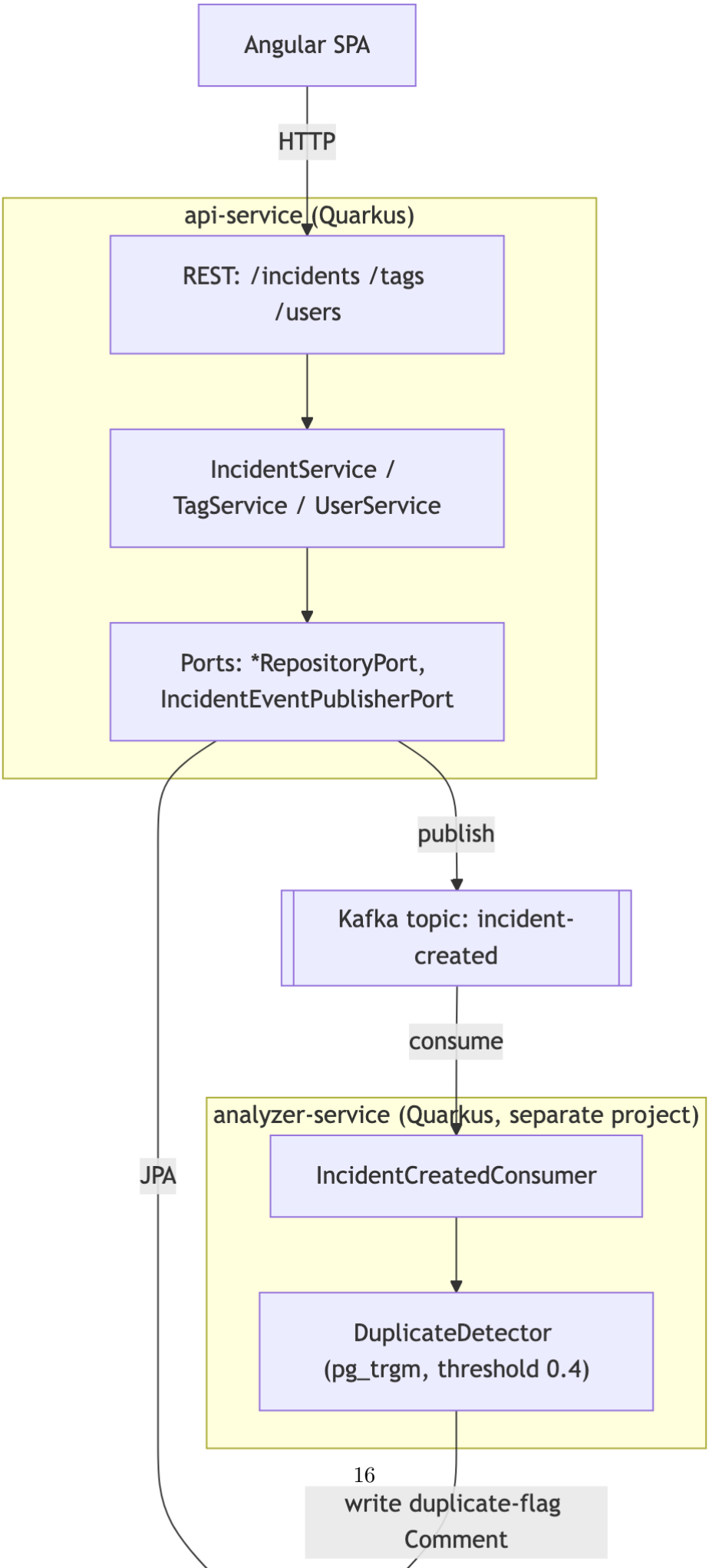


Figure 2: Application Architecture (Hexagonal)


Angular SPA
HTTP
api-service (Quarkus)
REST: /incidents /tags /users
IncidentService / TagService / UserService
Ports: *RepositoryPort, IncidentEventPublisherPort
publish
Kafka topic: incident-created
consume
analyzer-service (Quarkus, separate project)
IncidentCreatedConsumer
DuplicateDetector (pg_trgm, threshold 0.4)
JPA
write duplicate-flag Comment

The Application Architecture section provides the logical, macro-level view of the system, defining the architectural style (Hexagonal), the logical partitioning of services, and the design rules that govern layer interactions. It intentionally abstracts away concrete frameworks and deployment technologies. The Detailed Design section, later in the document, bridges this logical view to the concrete implementation: it specifies the technology stack (Quarkus, JAX-RS, JPA/Hibernate, Kafka, PostgreSQL), the framework-specific components, and the class-level execution of use cases and asynchronous processing.

7 Database Design

The schema is managed by Hibernate. The core relational facts include incidents referencing users (reporters), tags, and departments, while comments reference incidents and users (authors).

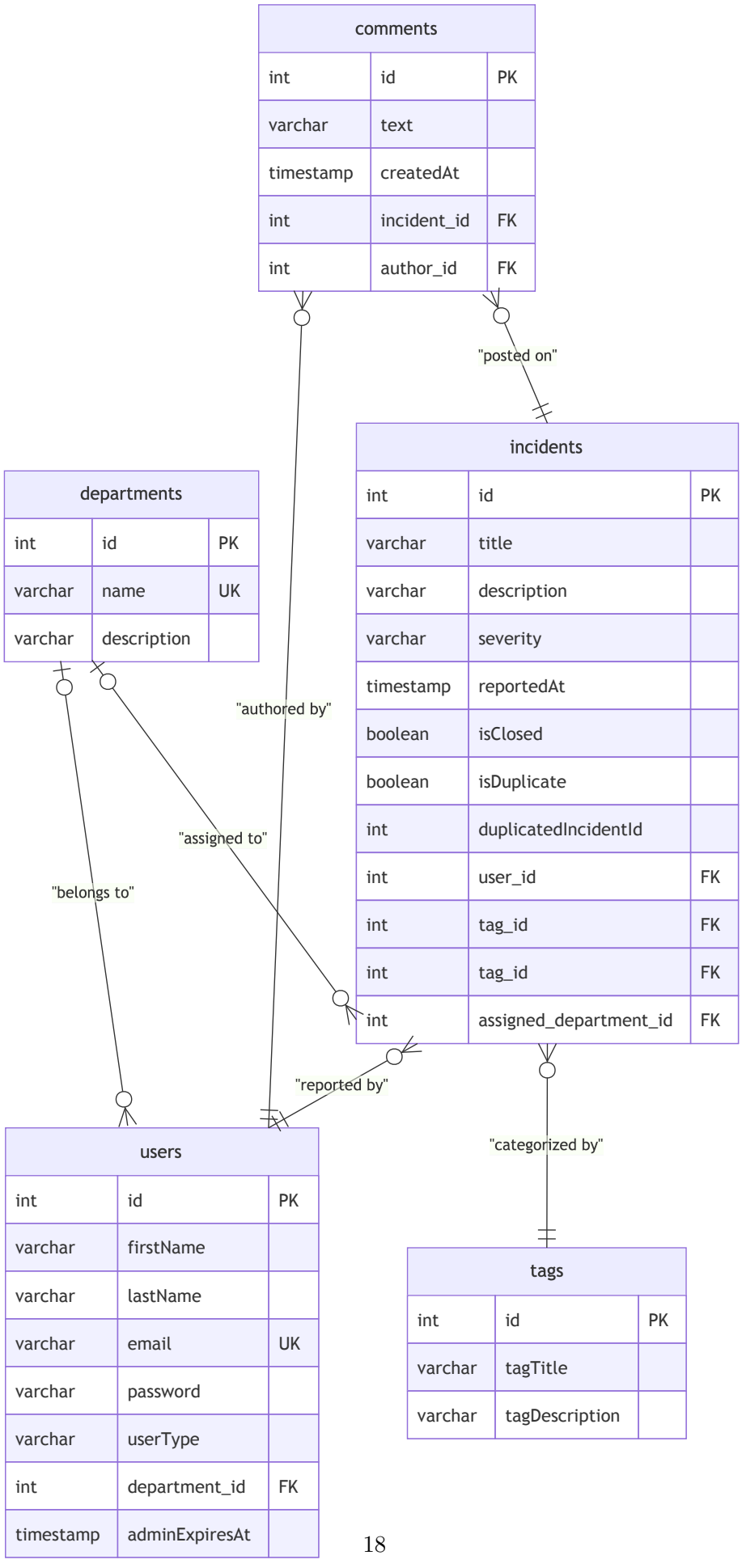


Figure 4: Database Entity-Relationship (ER) Diagram

8 Frontend Mockup Design

The user interface is built using Angular 19 with standalone components. The design adopts a modern, premium aesthetic (glassmorphism, dark themes) to ensure high user engagement.

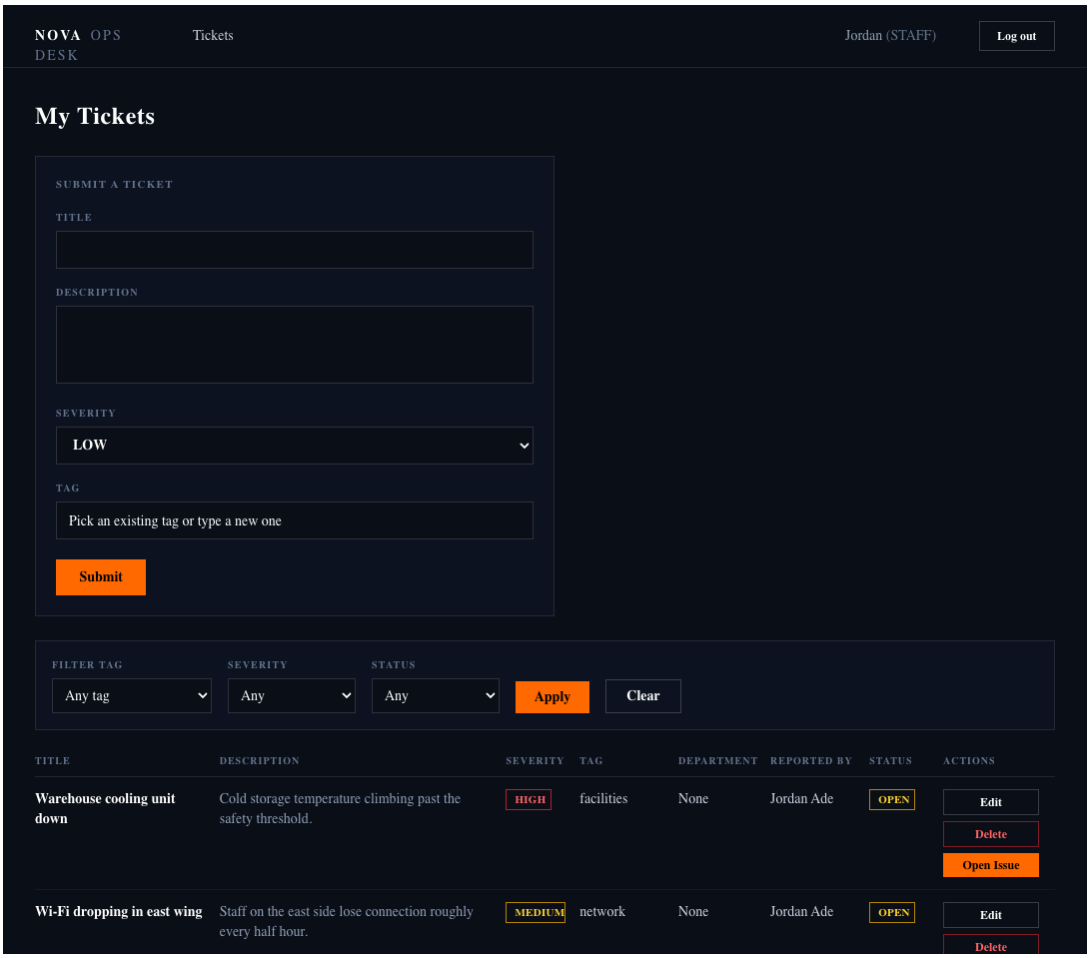


Figure 5: Mock 1: Dashboard

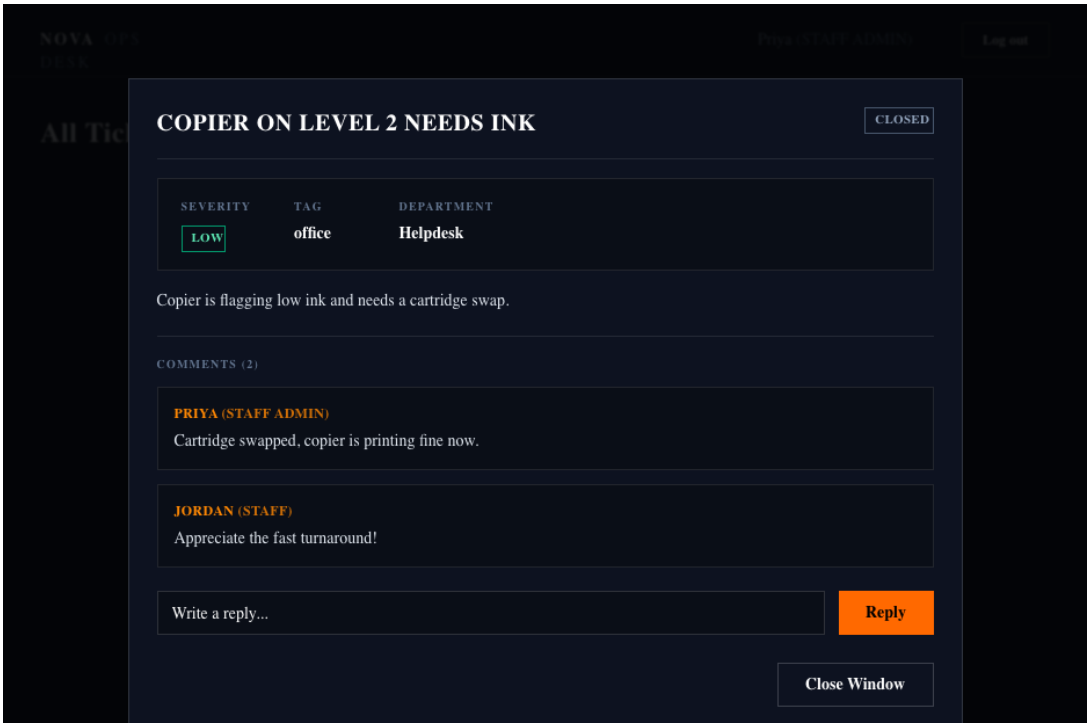


Figure 6: Mock 2: Comment View

- **Dashboard:** Displays a tabular view of incidents, color-coded by severity and status.
- **Modals:** A unified "GitHub Issue" style modal for viewing incident details and threaded comments.
- **Admin Controls:** Dedicated sections for managing tags, departments, and closing incidents.

9 Detailed Design

9.1 Description of the Main Components of the Software Implementation

- **Frontend (Angular):** Handles routing, route guards (`authGuard`, `adminGuard`), and state management. Communicates with the backend via REST services.
- **Backend API (Quarkus):** Implements the Hexagonal architecture claimed in §6. `adapters/in/rest` contains JAX RS resources; `application/service` contains `@Transactional` business logic; `adapters/out` contains JPA and Kafka implementations of domain ports.
- **Analyzer Service (Quarkus):** A lightweight consumer. Its driving adapter listens to the `incident created` Kafka topic, invokes the `pg_trgm` fuzzy matching query, and publishes its finding onto a second Kafka topic (`incident analysis completed`), which the backend consumes to mark the incident as a duplicate. No REST call exists between the two services, keeping them fully event driven in both directions.
- **PostgreSQL & Kafka:** Provide data persistence and message brokering, fully containerized alongside the services.

9.2 Hexagonal Layout on Disk

The project follows a pragmatic hexagonal (ports-and-adapters) structure that cleanly separates the core business logic from framework and infrastructure concerns. The directory layout mirrors the layers described in the architecture:

```
SWAM
|
|__ src
    |__ main
        |__ java
            |__ com
                |__ msohailse
                    |__ app
                        |__ incident
                            |__ adapters
                                |__ in
                                    |__ rest
                                |__ out
                                    |__ messaging
                                    |__ persistence
                            |__ application
                                |__ port
                                |__ service
                            |__ domain
        |__ resources
            |__ application.properties
```

- **domain** – Contains the pure domain entities (`Incident`, `User`, `Tag`, `Department`, `Comment`) and domain-specific enums. These classes have no dependencies on Quarkus or any other framework.
- **port** – Defines the ports (interfaces) that the core uses to communicate with the outside world, e.g. `RepositoryPort`, `IncidentEventPublisherPort`, `IncidentQueryPort`.
- **service** – Implements the hexagonal services (use-case interactor) such as `IncidentService` and `IncidentQueryService`. They depend only on the ports.
- **rest** – The JAX-RS adapters (controllers) that expose the HTTP API. They translate JSON payloads into DTOs and delegate to the application services.
- **persistence** – JPA/Hibernate adapters that implement `RepositoryPort`. Entity classes live here and map to the Postgres schema.
- **messaging** – Kafka adapters that implement `IncidentEventPublisherPort`. They use MicroProfile Reactive Messaging to emit and consume events.
- **application.properties** – Configuration for Quarkus, datasource, and Kafka channels.

```
analyzer_microservice
|__ src
    |__ main
        |__ java
        |   |__ com
        |       |__ msohailse
        |           |__ analyzer
        |               |__ consumer
        |               |   |__ IncidentCreatedConsumer.java
        |               |__ service
        |               |   |__ DuplicateDetector.java
        |               |__ client
        |               |   |__ ApiRestClient.java
        |__ resources
            |__ application.properties
```

- **consumer** – Listens on the Kafka topic for incident creation events and forwards them to the duplicate detection logic.
- **service** – Contains the DuplicateDetector class performing similarity calculations and publishing results.
- **client** – ApiRestClient communicates the detection outcomes back to the main API service via a secondary Kafka topic.

Zooming into the specific Java components, the `backend` package tree literally mirrors the domain/application/adapters split:

```
backend/src/main/java/.../incident/
|__ domain/              Incident, Tag, User, Department, Comment,
|                        Severity, UserType   (@Entity, Pragmatic Hexagonal)
|__ application/
|   |__ port/out/        IncidentRepositoryPort, IncidentEventPublisherPort,
|   |                    TagRepositoryPort, UserRepositoryPort,
|   |                    DepartmentRepositoryPort, CommentRepositoryPort
|   |                                              (interfaces only)
|   |__ service/         IncidentService, TagService, UserService,
|                        DepartmentService          (@Transactional)
|__ adapters/
    |__ in/rest/         IncidentResource, TagResource, UserResource,
    |                    DepartmentResource          (JAX RS)
    |__ in/messaging/    AnalysisCompletedConsumer   (@Incoming)
    |__ out/persistence/ IncidentPostgresRepository, ...
    |                    (implement the *RepositoryPort interfaces above)
    |__ out/messaging/   KafkaIncidentEventPublisher
                         (implements IncidentEventPublisherPort)
```

The dependency arrow only ever points inward: `adapters` depend on `application.port.out` interfaces, `application.service` depends on those same interfaces (never on a concrete adapter), and `domain` depends on nothing at all. This is what actually keeps `IncidentService` swappable, replacing Postgres or Kafka means writing a new class under `adapters/`, not touching a single line of business logic.

9.3 Mapping Architecture to Code

Below is how the conceptual hexagonal layers map to the actual Quarkus Java implementation.

1. The Web Traffic Cop (JAX-RS and Jackson)

```
@Path("/incidents")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class IncidentResource {

    @Inject
    IncidentService incidentService;

    @POST
    public Response createIncident(IncidentDTO request) {
        Incident incident = incidentService.create(request);
        return Response.ok(incident).build();
    }
}
```

2. The Core Logic (Hexagonal Service)

```
@ApplicationScoped
public class IncidentService {

    @Inject
```

```
    RepositoryPort repository;

    @Inject
    IncidentEventPublisherPort eventPublisher;

    @Transactional
    public Incident create(IncidentDTO dto) {
        Incident incident = new Incident();
        incident.setTitle(dto.getTitle());

        repository.save(incident);

        eventPublisher.publish(incident);

        return incident;
    }
}
```

3. The Database Bridge (JPA and Hibernate)

```
@Entity
@Table(name = "incidents")
public class Incident {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "title")
    private String title;

    @Column(name = "is_closed")
    private boolean isClosed;
}
```

4. The Event-Driven Flow (Kafka) Verbatim excerpt from `adapters/out/messaging/KafkaIncidentEv`

```
@ApplicationScoped
public class KafkaIncidentEventPublisher implements IncidentEventPublisherPort {

    @Inject
    @Channel("incident-created")
    Emitter<String> emitter;

    @Inject
    ObjectMapper objectMapper;

    public record IncidentCreatedEvent(int incidentId, String title,
            String description, String tagTitle) {}

    @Override
    public void publishIncidentCreated(int incidentId, String title,
```

```
            String description, String tagTitle) {
        String json = objectMapper.writeValueAsString(
                new IncidentCreatedEvent(incidentId, title, description, tagTitle));
        emitter.send(json);
    }
}
```

5. Lightweight CQRS (The Read Path) Verbatim excerpt from `adapters/out/persistence/IncidentP` — see §9.7 for the full three-class walkthrough (`IncidentResource.findAll()` → `IncidentService.findFilt` → this method):

```
public List<Incident> findFiltered(String tagTitle, Severity severity,
        Boolean closed, Department department, User reportedBy) {
    StringBuilder jpql = new StringBuilder("select i from Incident i where 1=1");
    if (tagTitle != null) {
        jpql.append(" and i.tag.tagTitle = :tagTitle");
    }
    // ... one more "and ... = :param" appended per non-null filter ...

    TypedQuery<Incident> query = em.createQuery(jpql.toString(), Incident.class);
    // ... setParameter(...) called only for the filters that were appended ...
    return query.getResultList();
}
```

Domain entities keep their `@Entity` annotations directly to leverage existing JPA mappings, while ports fully isolate persistence and messaging behind interfaces.

9.4 A Use Case Traced Through the Layers: Reporting an Incident

Concretely, a `POST /incidents` call moves through exactly these classes:

1. **`IncidentResource.create()`** (`adapters/in/rest`) deserializes the JSON body into a `CreateIncidentRequest` record and forwards it. It contains no business logic itself, only HTTP/JSON concerns.

2. **`IncidentService.create()`** (`application/service`) is where the actual rules live: it resolves the reporter through `UserRepositoryPort`, finds or creates the `Tag` through `TagRepositoryPort`, and builds a new `Incident`, whose own setters (`setTitle`, `setSeverity`, ...) enforce the domain's own invariants (non blank title, a real `Severity` enum value). The service knows only the two port *interfaces*, never that Postgres or Kafka sit behind them.

3. **`IncidentRepositoryPort.save()`** is implemented by **`IncidentPostgresRepository`** (`adapters/out/pe` the only class in the whole codebase that touches the JPA `EntityManager` for incidents.

4. Still inside the same `@Transactional` method, **`IncidentEventPublisherPort.publishIncidentCreate`** is implemented by **`KafkaIncidentEventPublisher`** (`adapters/out/messaging`), which serializes the event onto the `incident created` topic.

Nothing in this list needed to know about the other. The REST adapter never sees a `Tag`, the service never sees SQL, and the persistence adapter never sees Kafka.

9.5 How the Same Design Avoids Duplicates

The `incident created` event published in step 4 above is exactly the entry point for the two service deduplication pipeline described in §9.6: `analyzer service`'s own driving adapter consumes it, its `DuplicateDetector` runs the `pg_trgm` similarity check against other open incidents, and upon a match, publishes an `AnalysisCompletedEvent` onto `incident analysis completed`. Back on the API side, `AnalysisCompletedConsumer` (`adapters/in/messaging`) picks that up and calls `IncidentService.markDuplicate()`, which flags the incident and writes a system authored `Comment` in one transaction. The two services never call one another directly at any point in this loop. The same Publish Subscribe/Choreography style that created the incident in the first place is what catches its duplicate.

9.6 Asynchronous Duplicate Detection

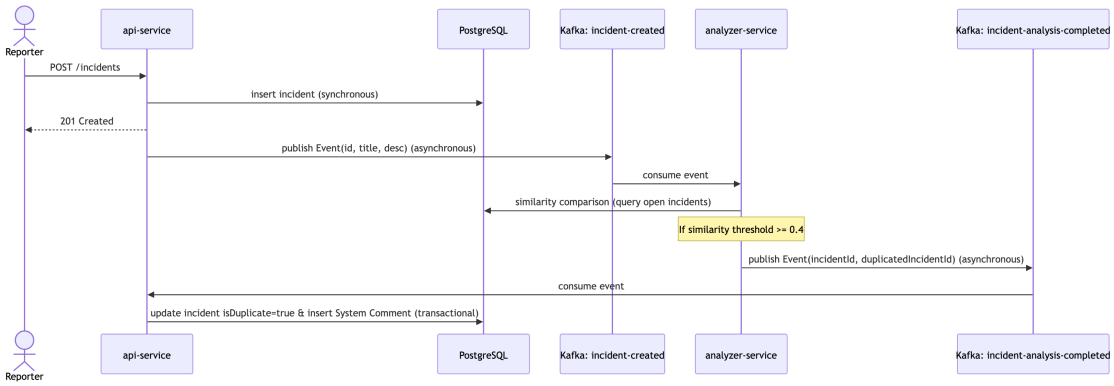


Figure 7: Duplicate Detection Sequence Diagram

Duplicate detection occurs asynchronously to keep the main write path non-blocking. When a new incident is created, the system publishes a message to a Kafka topic. An autonomous analyzer service processes the message and runs a similarity calculation. Rather than directly editing database tables, the analyzer service communicates its findings back to the API service using a secondary Kafka topic, maintaining clean architectural boundaries. Both directions of this exchange are plain publish/subscribe — the backend never calls the analyzer, and the analyzer never calls the backend; each just reacts to the topic it's listening on.

9.7 CQRS-Lite

A separate read path allows flexible filtering (`GET /incidents?tag=...`) using dynamic JPQL generation. This provides read flexibility without the overhead of event sourcing or a separate projection store.

Concretely, this is a single query path, entirely disjoint from the write path (`create`, `update`, or `close`), spanning exactly three classes:

1. `IncidentResource.findAll()` (`adapters/in/rest`) accepts `tag`, `severity`, `status`, and `actingUserId` as optional query parameters. Every role, including Reporter, Department Admin, and Super Admin, hits this exact same endpoint.

2. `IncidentService.findFiltered()` (`application/service`) executes exactly two tasks. First, it resolves the implicit scope of the caller via `resolveListScope()`. A Super Admin is unscoped, an active Department Admin is restricted to their own department, and anyone else is restricted to their own reports. Second, it forwards the tag, severity, and

status filters alongside that scope to the repository. The scope is derived purely from the `User` record of the caller, never from a client-supplied parameter. Therefore, a request can narrow what it sees but never widen it.

3. `IncidentPostgresRepository.findFiltered()` (`adapters/out/persistence`) builds the query with a plain `StringBuilder` over JPQL. It starts with `"select i from Incident i where 1=1"` and appends an `and i.field = :param` clause, binding it only for each filter that is actually non-null. There is no Criteria API, no Specifications, and no separate projection or read-model table. The read side queries the exact same `incidents` table that the write side writes to, simply through its own dedicated method.

This is deliberately the lite version of CQRS mentioned in Section 1. It provides read-side query flexibility without event sourcing or a denormalized read store. Both of these are listed under Future Developments in Section 11 as the natural next step if stricter read and write isolation were ever required.

10 Frontend Preview of the UI

The resulting UI delivers a cohesive, responsive experience.

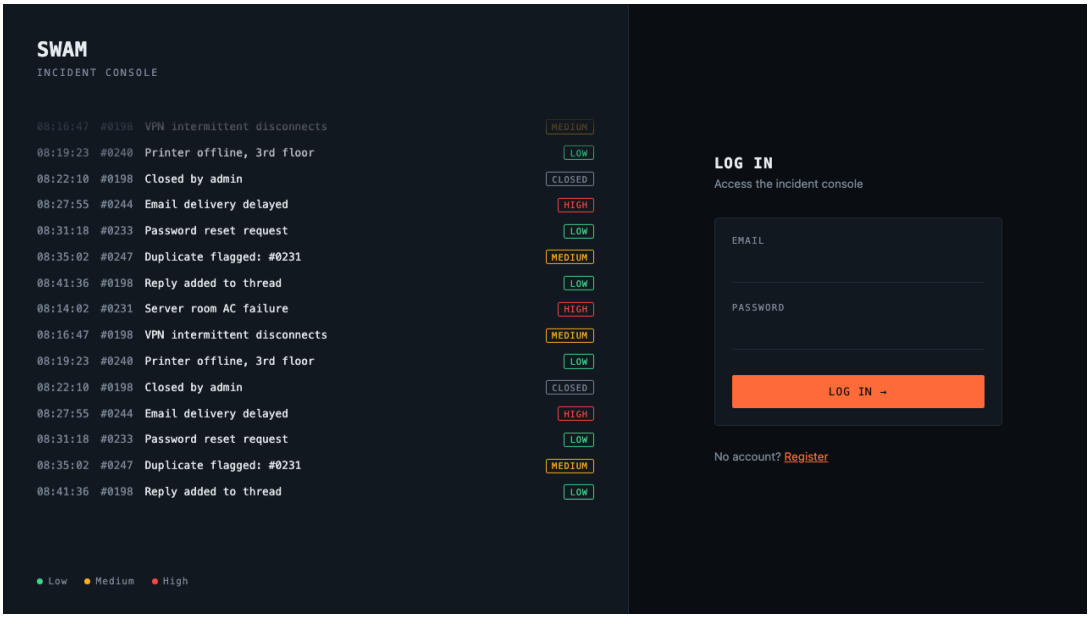


Figure 8: Authentication: Login

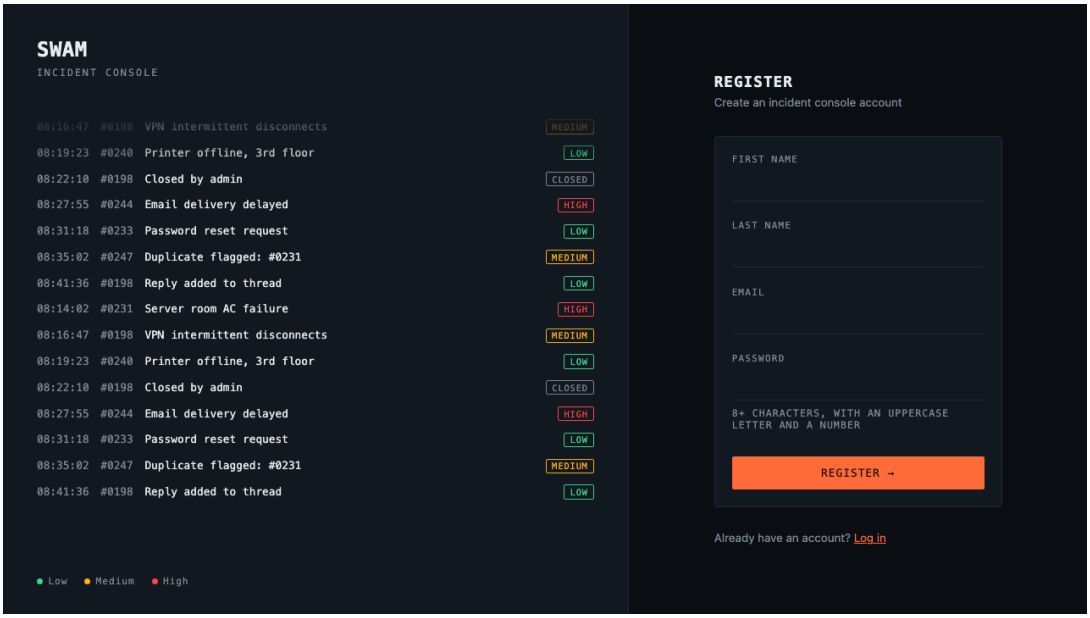


Figure 9: Authentication: Register

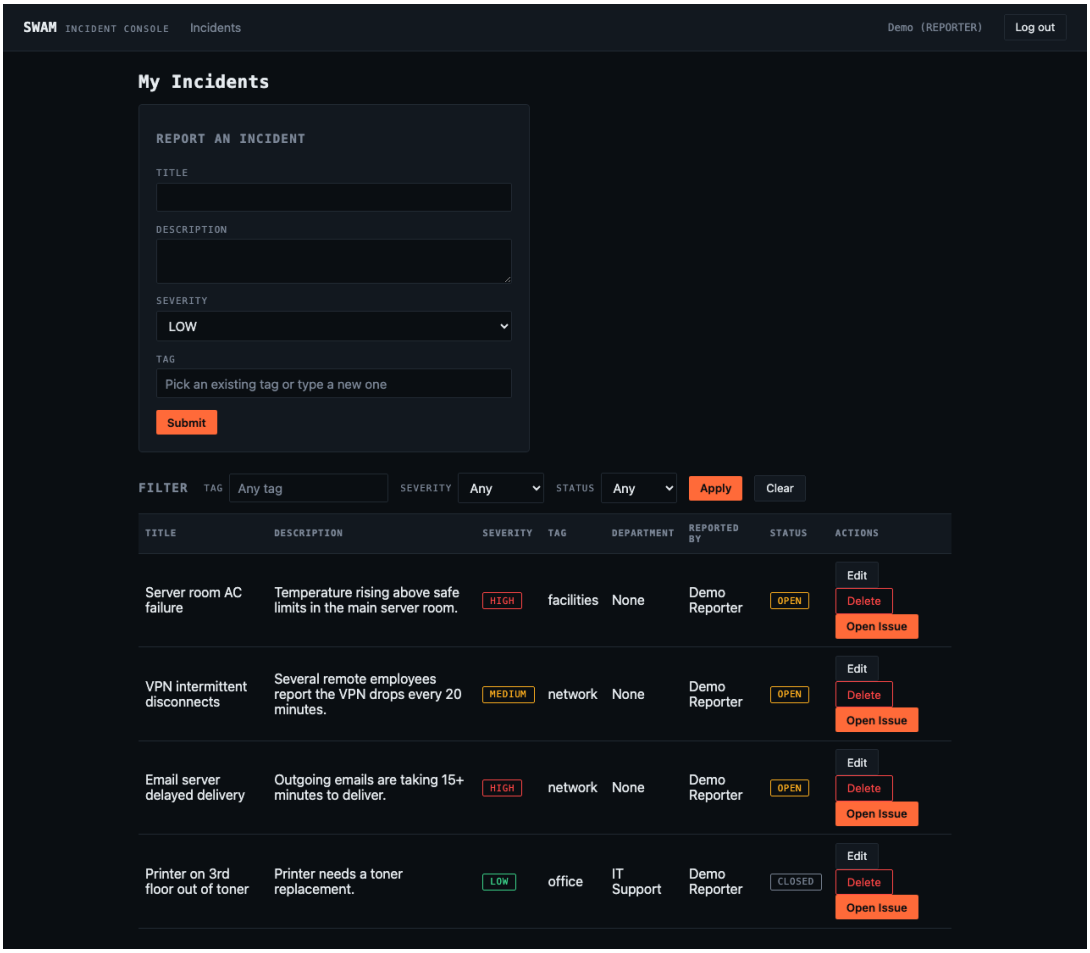


Figure 10: Reporter View: My Incidents

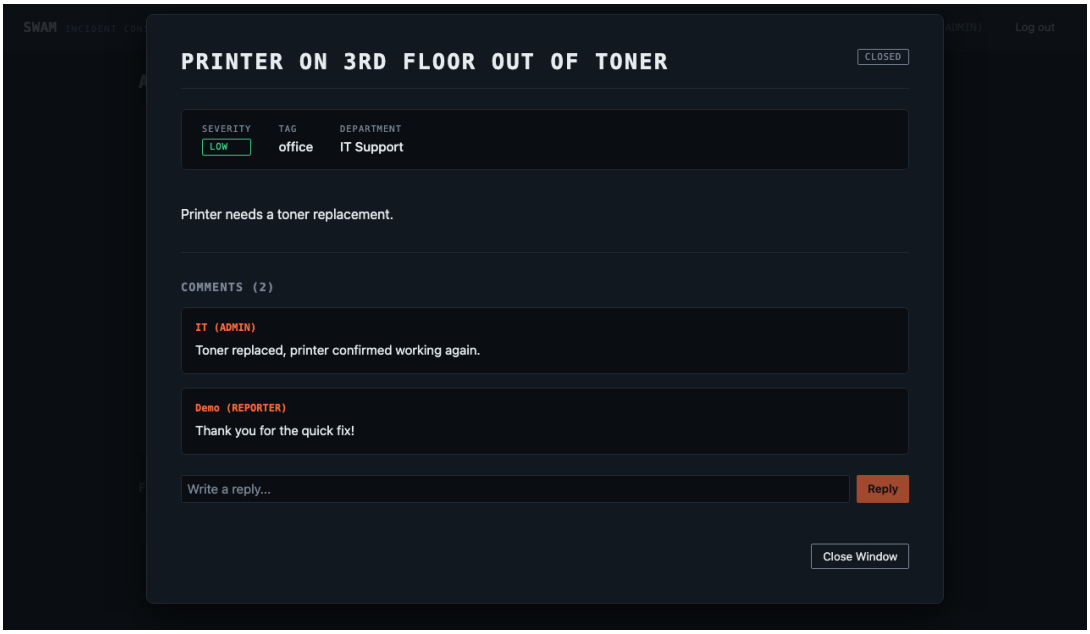


Figure 11: Admin View: Issue Details and Comment Thread

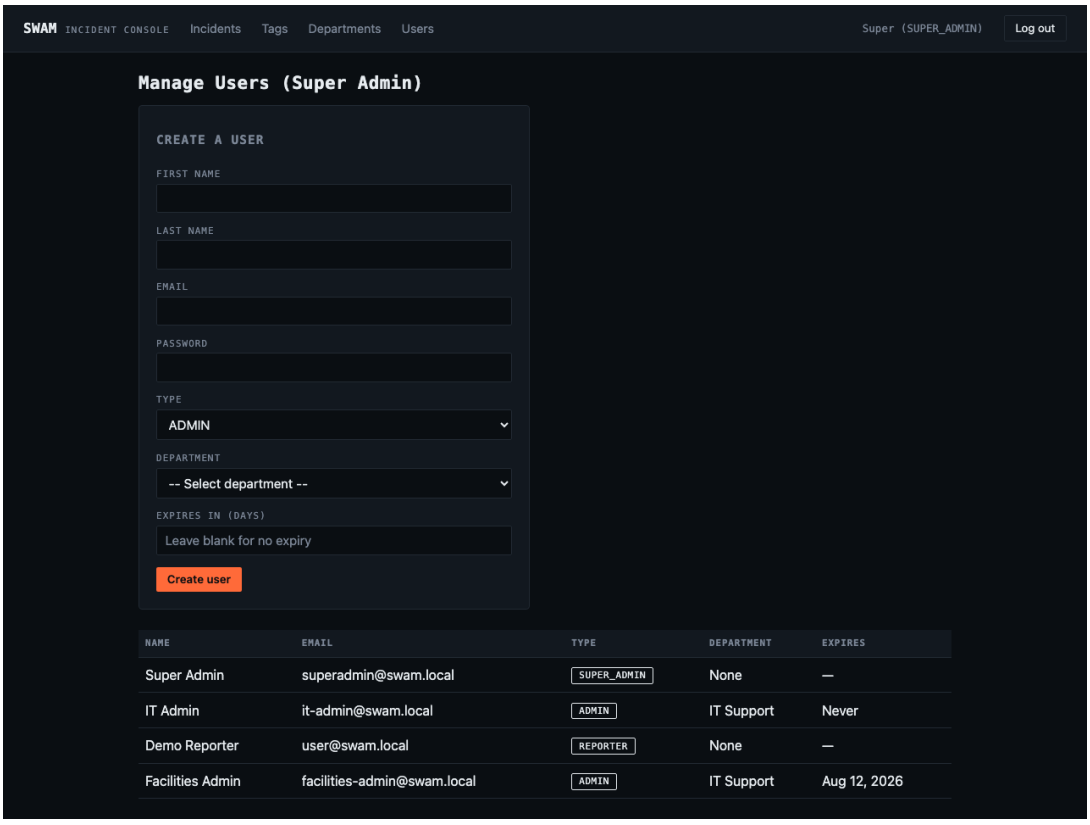


Figure 12: Super Admin View: Manage Users, showing a time-boxed `adminExpiresAt` on the "Facilities Admin" row versus a permanent ("Never") admin

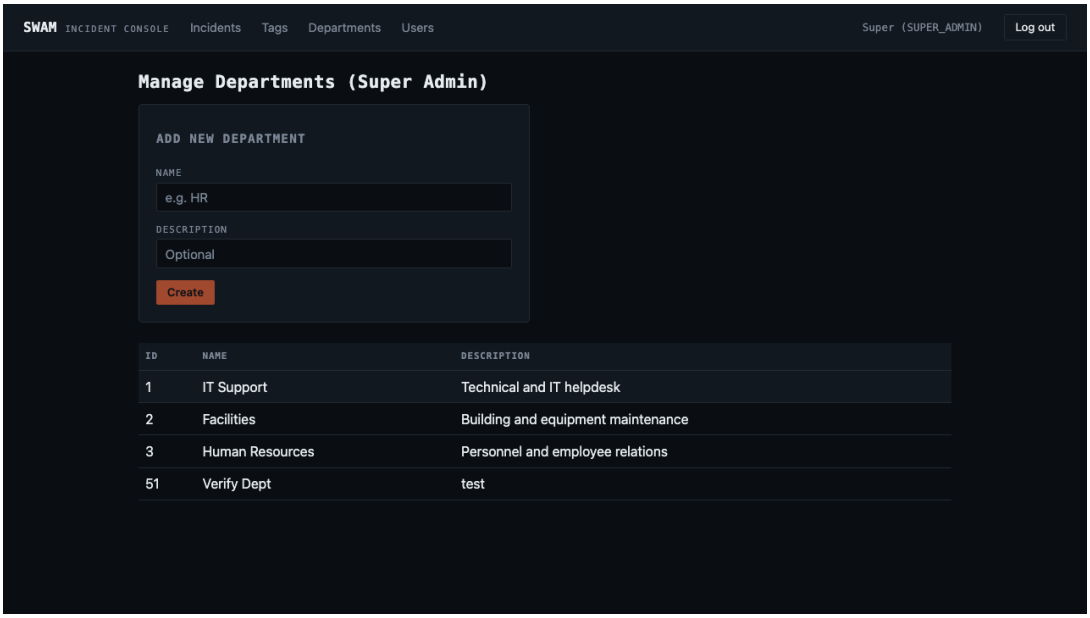


Figure 13: Super Admin View: Department Creation and Management

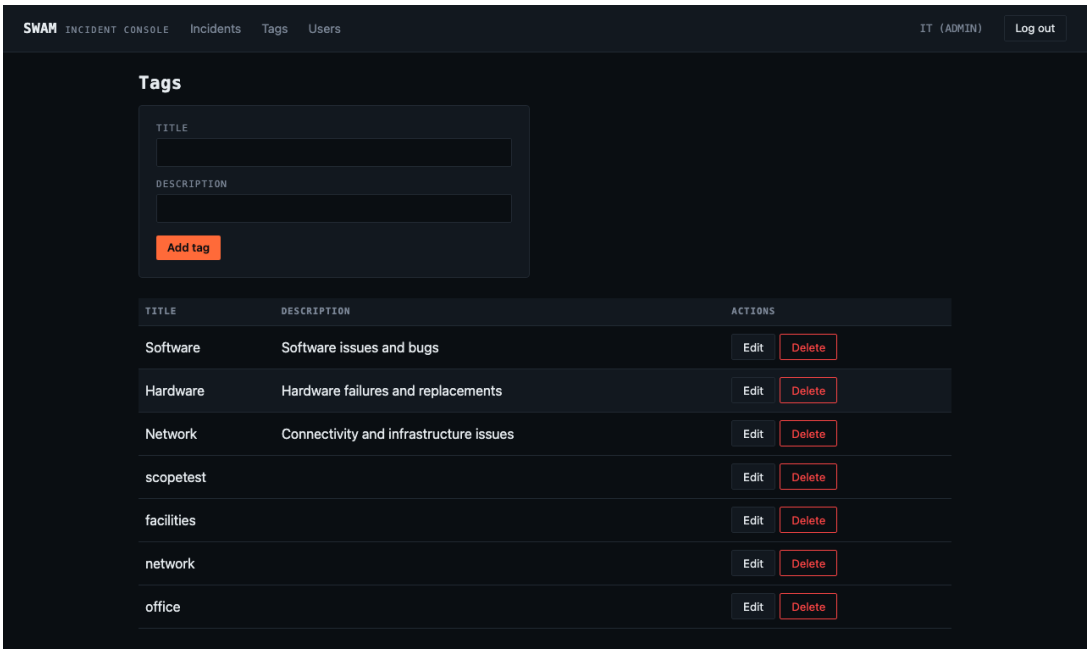


Figure 14: Admin View: Tag Creation and Management

11 Running the Application Locally

The system is fully containerized and can be launched using Docker Compose. From the root directory, navigate to the `deploy` folder and use the following commands:

```
cd deploy
cp .env.example .env
docker compose up -d --build
```

This will spin up PostgreSQL, Kafka, the Quarkus Backend API, the Analyzer Service, and the Angular Frontend. The web application will be available at `http://localhost:4200`.

12 Future Developments and Conclusions

12.1 Future Developments

- **Full CQRS Read Models:** Denormalizing data into a dedicated read projection updated entirely from Kafka events.
- **Semantic Duplicate Detection:** Integrating `pgvector` and sentence embeddings for better duplicate detection accuracy.

12.2 Conclusions

This prototype successfully demonstrates the integration of Hexagonal Architecture, Event-Driven Architecture, and CQRS in a modern stack. The separation of the synchronous API from the asynchronous Analyzer Service highlights the benefits of decoupled scaling. By containerizing the solution and validating it via Kubernetes Horizontal Pod Autoscaling (HPA), the project exceeds basic design requirements, offering a fully deployable, scalable operative scenario for incident management.